



UNIVERSIDAD PONTIFICIA DE SALAMANCA
FACULTAD DE INFORMÁTICA
Máster Universitario en Informática Móvil

Proyecto Fin de Máster

Reactive programming para Android

Jorge Sanzo Hernando

 Firmado digitalmente
por BERJON
GALLINAS ROBERTO -
09776013R
Fecha: 2022.10.12
13:51:54 +02'00'

Dr. Roberto Berjón Gallinas
Director

Salamanca, Octubre de 2022

Resumen

Este Trabajo de Fin de Máster consiste en la investigación de las diferencias, ventajas, e inconvenientes que ofrece la programación Reactiva frente a la programación Funcional.

De forma resumida la programación Reactiva es un paradigma de programación orientado al trabajo con flujos de datos finitos o infinitos de manera asíncrona, está relacionado con el patrón de diseño Observer. Este patrón se centra en que hay un objeto que está siendo observado y cuando cambia de estado, el resto son notificados y actualizados para después realizar las operaciones que se deseen en función de ese cambio de estado.

Por otro lado, la programación Funcional es un paradigma orientado a las funciones, se pueden almacenar funciones en variables e incluso usarlas como parámetros tanto de salida como de entrada de otras funciones, se centra sobre todo en que se quiere hacer y no en cómo y no tiene en cuenta los estados de los objetos, sería prácticamente lo contrario a la programación reactiva.

Para poder observar mejor los resultados y tener varios casos prácticos, se ha desarrollado una aplicación móvil en Kotlin donde vamos a poder interactuar con una base de datos realizando operaciones CRUD y observando como lo gestiona cada paradigma, jugar con los sensores de datos en tiempo real del dispositivo (Sensor de Aceleración, Sensor de Orientación y Sensor de Luminosidad) viendo cómo se recogen y muestran los datos de ambas formas y por ultimo ejecutar operaciones matemáticas, de filtrado, mapeo y ordenación sobre grandes colecciones de datos utilizando listas y secuencias.

Abstract

This Master's Thesis consists of investigating the differences, advantages and disadvantages that Reactive programming offers compared to Functional programming.

In short, Reactive programming is a programming paradigm oriented to work with finite or infinite data flows asynchronously, it is related to the Observer design pattern. This pattern focuses on the fact that there is an object that is being observed and when it changes the state, the others are notified and updated performing the desired operations based on that state change.

On the other hand, Functional programming is a function-oriented paradigm, you can store functions in variables and even use them as output or input parameters of other functions, it focuses on what you want to do and not on how to do. It does not consider the states of the objects, it would be the opposite of reactive programming.

In order to observe the results and have several practical cases, a mobile application has been developed in Kotlin where we will be able to interact with the local database by performing CRUD operations and observe how each paradigm manages it, playing with the device sensors data in real time (Acceleration Sensor, Orientation Sensor and Luminosity Sensor) seeing how the data is collected and displayed in both ways and finally executing mathematical, filtering, mapping and ordering operations on large data collections using lists and sequences.

Descriptores

Reactive Programming, Functional Programming, Mobile Application, Android, Kotlin

Índice

1.	Introducción	9
2.	Antecedentes	10
3.	Descripción y Requisitos	12
3.1	Descripción de la aplicación	12
3.2	Requisitos funcionales	17
3.3	Requisitos no funcionales	18
4.	Desarrollo	20
4.1	Arquitectura	20
4.2	Otros aspectos importantes.....	26
5.	Resultados y conclusiones.....	27
5.1	Operaciones sobre bases de datos	27
5.2	Datos en tiempo real.....	31
5.3	Secuencias y listas	34
6.	Líneas de trabajo futuro.....	39
6.1	Investigación	39
6.2	Estudio de datos.....	39
	Manual de usuario	42

Índice de Figuras

Figura 3-1 Database Model.....	13
Figura 4-1 Project Package Model 1	23
Figura 4-2 Project Package Model 2	23
Figura 4-3 Project Package Model 3	24
Figura 4-4 Project Package Model 4	24
Figura 4-5 Project Package Model General.....	25
Figura 5-2 Database Create Team.....	28
Figura 5-1 Database Read	28
Figura 5-3 Database Update Team	29
Figura 5-4 Database Synchronize.....	29
Figura 5-5 Database Delete Team.....	30
Figura 5-7 Sensors High Interval	32
Figura 5-6 Sensors Running.....	32
Figura 5-8 Sensors Low Interval.....	33
Figura 5-9 Collections One Execution	35
Figura 5-10 Collections Two Executions	35
Figura 5-11 Collections List Data.....	36
Figura 5-12 Collections Sequence Data	36
Figura 5-13 Collections Avg Data	37
Figura 5-14 Collections All Data	37
Figura 6-1 Home Screen.....	42
Figura 6-2 Navigation Menu	43
Figura 6-3 Database Menu.....	44
Figura 6-4 Database Screen	44
Figura 6-5 Sensors Screen	45
Figura 6-6 Collections Screen.....	46

Índice de Tablas

Tabla 1 - Casos de Prueba	15
---------------------------------	----

1. Introducción

La tecnología avanza cada vez más rápido y es indudable que, hoy en día, hay que estar siempre al corriente de las nuevas tendencias, es por eso por lo que he encontrado realmente interesante el estudio e investigación de la programación reactiva, ya que es algo bastante reciente.

Por otra parte, hace tiempo, me encontré con el patrón Observer, algo que al principio no entendí muy bien ni le veía la utilidad por ningún lado pero que tiempo después me solucionó muchos problemas en numerosos proyectos en forma de “Callbacks” y empezó a llamarme la atención, aunque lo veía aun algo un poco complicado, más adelante cuando conocí Kotlin y la programación de Aplicaciones móviles, se me abrió un nuevo mundo de posibilidades con la programación reactiva ya que era mucho más sencilla de utilizar que en otros lenguajes y solucionaba muchos problemas relacionados con los datos en tiempo real, actualizaciones de estados de las vistas, peticiones a servicios web, etc.

También hacer mención especial al curso que realizó el profesor Antonio Leiva sobre programación reactiva dentro del curso de Desarrollo de Aplicaciones Móviles del Máster MIMO de la Universidad Pontificia de Salamanca ya que me ha sido de gran ayuda en el desarrollo del proyecto (1)

Es por esto por lo que decidí realizar este proyecto, con el objetivo de investigar las diferencias entre la programación funcional y la programación reactiva en diversos casos como, por ejemplo, realizar operaciones sobre una base de datos, gestionar productores de datos en tiempo real o las operaciones sobre colecciones de datos muy grandes, para después poder sacar conclusiones sobre cual es mejor en cada caso, cual es más eficiente o cual tarda menos tiempo en ejecutarse

Para ello, se va a estudiar el uso de Flow y Corrutinas para realizar operaciones CRUD sobre una base de datos, comparándolo con el uso de Listas normales y AsyncTask

Después, se va a hacer una comparación al recoger datos generados en tiempo real con los sensores del dispositivo, se comparará Flow y LiveData con Listas y el método de Polling, que consiste en la consulta activa de un valor cada determinado tiempo

Por último, se van a realizar una gran cantidad de operaciones sobre Secuencias y Listas para poder comparar con varios casos de prueba el desempeño de cada una en cuanto a tiempo de ejecución

2. Antecedentes

El matemático John McCarthy comenzó a desarrollar la programación funcional en torno a 1956 para IBM, fue utilizado en sus primeros proyectos de inteligencia artificial. Durante este desarrollo, se crea el lenguaje de programación LISP en 1958, siendo este el primer lenguaje de programación funcional basado en el cálculo lambda y el segundo lenguaje de programación de alto nivel más antiguo. Tiempo después evoluciona y surgieron nuevos lenguajes de programación que adoptaron este paradigma como Scala o Haskell que es puramente funcional. Actualmente hay muchos lenguajes de programación que permiten usar este paradigma, aunque no sean puro como por ejemplo Javascript, Java a partir de Java 8 o Kotlin (2)

La programación reactiva se empieza a desarrollar en 2010 y en 2014 se publica el “Reactive Manifesto” (3) (4) donde se definieron los estándares de este nuevo paradigma. El manifiesto indica que los sistemas reactivos deben ser responsivos, resilientes, elásticos y orientados a mensajes. Este nuevo tipo de programación se centra en el flujo asíncrono de datos ya sean finitos o infinitos, posibilitando también el flujo o stream de eventos. Estos eventos deben tener siempre como resultado un valor o un error.

Los principales lenguajes de programación reactiva son por ejemplo RxJava, BaconJs, RxNET o Kotlin, no son lenguajes de programación estrictamente nuevos si no adaptaciones de los ya existentes con nuevas librerías para facilitar el uso y aprendizaje de esta nueva tecnología

Para este proyecto vamos a centrarnos en los siguientes conceptos utilizados en el lenguaje de programación Kotlin

- LiveData: Es un tipo de datos observable, es decir, cada vez que cambiamos su valor con la función “postValue()” todos los observadores reciben la actualización y actúan en consecuencia. Está condicionado por el ciclo de vida del observador, si todos dejan de observarlo o todos están pausados, dejara de emitir cambios hasta que tenga un nuevo observador, esto es muy importante para evitar los “Memory Leaks”. Otra ventaja es que los datos siempre están actualizados ya que siempre se reciben los más nuevos, aunque el observador este inactivo, el LiveData almacena siempre el dato más reciente para emitirlo después (5)
- Flow: Es un tipo de datos que emite valores de forma secuencial, además pueden hacerlo de forma asíncrona por lo que trabajan muy bien con las Corrutinas. Hay tres tipos de actores involucrados en estas transmisiones de datos, el productor de datos que genera los valores, el intermediario que puede modificar los valores

durante el flujo y el consumidor que es el que utiliza finalmente esos valores. Poder modificar estos valores en mitad del flujo es muy útil para mapear campos de una respuesta web por ejemplo (6)

- Corrutinas: Son funciones que permiten ejecutar código de forma asíncrona, son muy útiles en operaciones que, de otra manera, bloquearían el hilo principal hasta que el resultado está listo, por ejemplo, las llamadas a una API. La principal ventaja son la ligereza de las operaciones, podemos ejecutar varias Corrutinas en el mismo subproceso ya que este no se bloquea y podemos ahorrar bastante memoria. Otra ventaja es que no hay “Memory Leaks” porque se ejecutan dentro de un “Scope” acotado de donde no salen, también es importante saber que, si lanzamos varias de forma encadenada, la cancelación de cualquiera supone la cancelación de todas las demás, lo que hace que sean más fáciles de gestionar (7)
- Sequence: Es un tipo de colección de datos cuya característica principal es que es de tipo “Lazy evaluate”, es decir, sus valores no son calculados hasta que son necesarios, esto hace que se necesite menos memoria que en las colecciones normales, permitiendo incluso manejar colecciones infinitas de datos. Hay que tener en cuenta que para las secuencias existe las operaciones “stateless” donde solo necesitan el elemento actual para ser realizadas, por ejemplo, cambiar su tipo o evaluar si es par o no y operaciones “stateful” que necesitan los demás elementos para poder ser ejecutadas, por ejemplo, reordenar la colección. Cuando realizamos las operaciones “stateless” se ejecutan sobre cada elemento hasta que se llega a una operación “stateful”, se espera a que todas terminen y genera el resultado final (8)
- StateFlow: Es un tipo de datos muy parecido a Flow con la particularidad de que sirve para gestionar los estados de una vista, para ello la vista debe observar el StateFlow que contenga su ViewModel, de esta manera cuando sea necesario, el ViewModel actualizará el estado de la vista y esta recibirá la actualización. La principal diferencia con Flow es que puede recopilar datos a pesar de que no haya ningún observador activo, siempre va a estar en memoria preparado para que alguien le consulte, solo desaparece si todas sus referencias a él en memoria desaparecen. Si el StateFlow está inactivo y un observador vuelve a observarle de nuevo, obtendrá el ultimo estado y también todos los anteriores que no han sido consumidos (9)

3. Descripción y Requisitos

3.1 Descripción de la aplicación

Para poder comparar ambos tipos de programación y poder sacar las conclusiones necesarias, la aplicación deberá realizar las siguientes funciones

- Operaciones CRUD:

Para la programación reactiva, se utilizan Corrutinas y Flow, esto permite que cuando el usuario pide los datos por primera vez, se crea un canal o flujo que se mantiene abierto, informando de cada actualización, de esta manera el usuario no se pierde ningún cambio cada vez que crea, borra o edita un equipo

En cambio, con la programación funcional, se utilizan Listas y AsyncTask, esto provoca que cuando el usuario pide los datos por primera vez, se devuelven desde la base de datos y el flujo o canal se cierra, esto provoca que el usuario no pueda saber si ha habido cambios o no en la base de datos hasta que vuelva a pedir los valores de forma manual

El esquema de la base de datos es el siguiente:

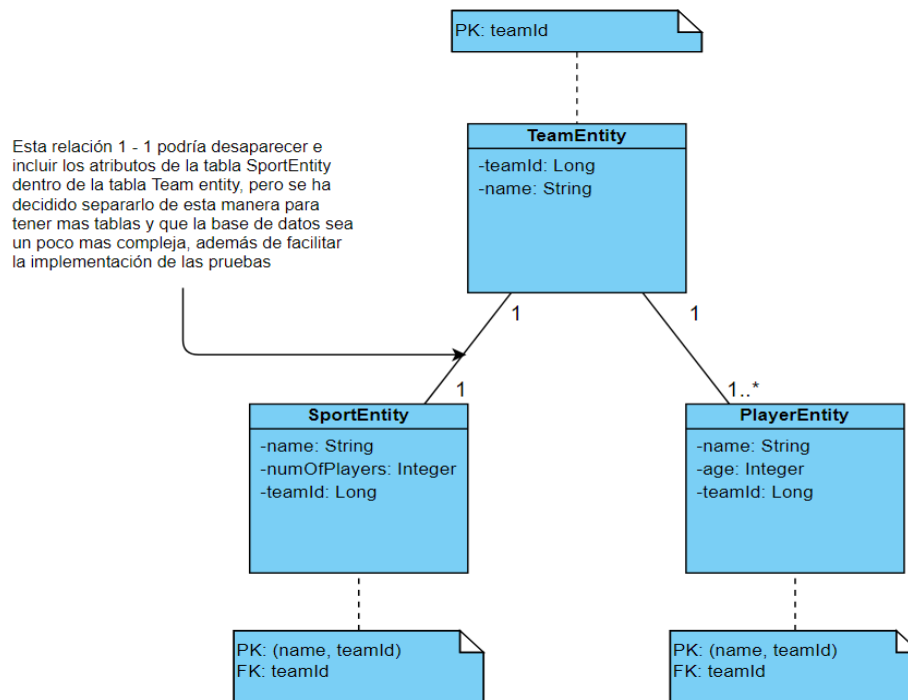


Figura 3-1 Database Model

Las operaciones necesarias para las pruebas serán gestionadas por la librería Room (10) y son las siguientes:

- Guardar un deporte:

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
abstract suspend fun saveSport(sportEntity: SportEntity)
```

- Guardar un jugador:

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
abstract suspend fun savePlayer(playerEntity: PlayerEntity)
```

- Guardar un equipo:

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
abstract suspend fun saveTeam(teamEntity: TeamEntity)
```

- Borrar todos los deportes:

```
@Query("delete from Sport")
abstract suspend fun deleteSports()
```

- **Borrar todos los jugadores:**

```
@Query("delete from Player")
abstract suspend fun deletePlayers()
```

- **Borrar todos los equipos:**

```
@Query("delete from Team")
abstract suspend fun deleteTeams()
```

- **Obtener los equipos de forma Reactiva**

```
@Query("select * from Team")
abstract fun getTeamsReactive(): Flow<List<TeamEntity>>
```

- **Obtener el deporte de un equipo de forma Reactiva**

```
@Query("select * from Sport s where s.team_ID == teamId")
abstract fun getSportReactive(teamId: Long): Flow<SportEntity>
```

- **Obtener los jugadores de un equipo de forma Reactiva**

```
@Query("select * from Player p where p.team_ID == teamId")
abstract fun getPlayersReactive(teamId: Long):
Flow<List<PlayerEntity>>
```

- **Obtener los equipos de forma Funcional**

```
@Query("select * from Team")
abstract fun getTeamsFunctional(): List<TeamEntity>
```

- **Obtener el deporte de un equipo de forma Funcional**

```
@Query("select * from Sport s where s.team_ID == teamId")
abstract fun getSportFuncitonal(teamId: Long): SportEntity
```

- **Obtener los jugadores de un equipo de forma Funcional**

```
@Query("select * from Player p where p.team_ID == teamId")
abstract fun getPlayersFunctional(teamId: Long):
List<PlayerEntity>
```

- **Actualizar el nombre de un equipo de la base de datos**

```
@Query("update Team set name = newTeamName where team_id =
teamId")
abstract suspend fun updateTeam(teamId: Long, newTeamName:
String)
```

- **Borrar un equipo de la base de datos**

```
@Query("delete from Team where team_id = teamId")
abstract suspend fun removeTeam(teamId: Long)
```

- Sensores del dispositivo:

En este caso también se ha utilizado Flow para la programación reactiva para mantener abierto ese canal de emisión de datos que son recogidos de forma constante por los sensores del móvil

Se han utilizado los siguientes sensores del dispositivo (11) para realizar las pruebas:

- `AndroidSensor.TYPE_LIGHT`: Mide el nivel de iluminación en el ambiente
- `AndroidSensor.TYPE_ROTATION_VECTOR`: Mide la orientación del dispositivo en los tres ejes de rotación X, Y, Z
- `AndroidSensor.TYPE_LINEAR_ACCELERATION`: Mide la fuerza de aceleración del dispositivo en los tres ejes X, Y, Z descontando la gravedad

Del mismo modo, para la programación funcional se han utilizado Listas normales y el método de Polling, esto consiste en consultar el valor de una variable de forma constante esperando un determinado tiempo entre consultas, este tiempo puede ser variable

- Operaciones pesadas:

En este caso se quiere comparar diferencia de tiempo al ejecutar una serie de operaciones sobre grandes colecciones de datos, en concreto sobre Secuencias y Listas (8) para poder observar sus diferencias, las operaciones siguen el siguiente orden:

1. Seleccionar los N primeros números de la colección
2. Obtener los M primeros números impares
3. De ellos escoger solo los primos
4. Calcular el cuadrado de cada uno
5. Convertir los números a String
6. Escoger aquellos con menos de D dígitos de longitud
7. Volver a convertirlos a Int
8. Ordenar los números de mayor a menor

Como podemos observar hay varios parámetros para tener en cuenta, la variación de estos en los diferentes casos de prueba hará que se obtengan diferentes resultados para colecciones más grandes o pequeñas

Tabla 1 - Casos de Prueba

N.º de la prueba	Longitud lista	N	M	D
1	1.000.000	50.000	1.000	7
2	1.000.000	50.000	1.000	8
3	1.000.000	50.000	1.000	9

4	1.000.000	60.000	1.000	7
5	1.000.000	60.000	1.000	8
6	1.000.000	60.000	1.000	9
7	1.000.000	70.000	1.000	7
8	1.000.000	70.000	1.000	8
9	1.000.000	70.000	1.000	9
10	1.000.000	80.000	1.000	7
11	1.000.000	80.000	1.000	8
12	1.000.000	80.000	1.000	9
13	1.000.000	90.000	1.000	7
14	1.000.000	90.000	1.000	8
15	1.000.000	90.000	1.000	9
16	1.000.000	100.000	1.000	7
17	1.000.000	100.000	1.000	8
18	1.000.000	100.000	10.000	9
19	1.000.000	200.000	10.000	7
20	1.000.000	200.000	10.000	8
21	1.000.000	200.000	10.000	9
22	1.000.000	300.000	10.000	7
23	1.000.000	300.000	10.000	8
24	1.000.000	300.000	10.000	9
25	1.000.000	400.000	10.000	7
26	1.000.000	400.000	10.000	8
27	1.000.000	400.000	10.000	9
28	1.000.000	500.000	10.000	7

29	1.000.000	500.000	10.000	8
30	1.000.000	500.000	10.000	9

3.2 Requisitos funcionales

- RF-01: La aplicación deberá mostrar al usuario la vista principal nada más iniciar la aplicación
- RF-02: La aplicación permitirá al usuario navegar entre las vistas de gestión de base de datos, gestión de datos en tiempo real y gestión de grandes colecciones de datos
- RF-03: La aplicación deberá mostrar al usuario los equipos existentes en la base de datos
- RF-04: La aplicación deberá permitir al usuario crear un nuevo equipo
- RF-05: La aplicación deberá permitir al usuario modificar los equipos existentes
- RF-06: La aplicación deberá permitir al usuario borrar un equipo existente
- RF-07: La aplicación deberá permitir al usuario sincronizar la lista de equipos con los equipos de la base de datos cuando esta se quede desactualizada
- RF-08: La aplicación deberá mostrar al usuario el nivel actual de luminosidad en tiempo real
- RF-09: La aplicación deberá mostrar al usuario el nivel máximo de luminosidad registrado
- RF-10: La aplicación deberá mostrar al usuario el nivel mínimo de luminosidad registrado
- RF-11: La aplicación deberá mostrar al usuario el nivel actual del sensor de orientación en tiempo real
- RF-12: La aplicación deberá mostrar al usuario el nivel máximo registrado del sensor de orientación
- RF-13: La aplicación deberá mostrar al usuario el nivel mínimo registrado del sensor de orientación
- RF-14: La aplicación deberá mostrar al usuario el nivel actual del sensor de aceleración en tiempo real
- RF-15: La aplicación deberá mostrar al usuario el nivel máximo registrado del sensor de aceleración
- RF-16: La aplicación deberá mostrar al usuario el nivel mínimo registrado del sensor de aceleración
- RF-17: La aplicación deberá mostrar al usuario los datos totales recogidos por los sensores del dispositivo
- RF-18: La aplicación deberá mostrar al usuario los datos totales recogidos de forma funcional
- RF-19: La aplicación deberá mostrar al usuario los datos totales recogidos de forma reactiva
- RF-20: La aplicación deberá mostrar el porcentaje de efectividad de la recogida de datos de ambas formas, tanto funcional como reactiva

- RF-21: La aplicación deberá permitir al usuario cambiar la frecuencia de recogida de datos de forma funcional
- RF-22: La aplicación deberá permitir al usuario pausar la recogida de todos los tipos de datos
- RF-23: La aplicación deberá permitir al usuario reanudar la recogida de todos los tipos de datos
- RF-24: La aplicación deberá permitir al usuario reiniciar y borrar todos los datos recogidos
- RF-25: La aplicación deberá permitir al usuario ejecutar los casos de prueba de operaciones pesadas de forma funcional
- RF-26: La aplicación deberá permitir al usuario ejecutar los casos de prueba de operaciones pesadas de forma reactiva
- RF-27: La aplicación deberá mostrar al usuario el tiempo real de la ejecución de cada caso de prueba realizado con secuencias
- RF-28: La aplicación deberá mostrar al usuario el tiempo medio de la ejecución de cada caso de prueba realizado con secuencias
- RF-27: La aplicación deberá mostrar al usuario el tiempo real de la ejecución de cada caso de prueba realizado con listas
- RF-28: La aplicación deberá mostrar al usuario el tiempo medio de la ejecución de cada caso de prueba realizado con listas
- RF-29: La aplicación deberá permitir al usuario alternar entre todos los tiempos recogidos, ocultándolos o mostrándolos para cada tipo de ejecución

3.3 Requisitos no funcionales

- RNF-01: Los equipos tendrán nombre, deporte asociado y número de jugadores
- RNF-02: La aplicación utilizará la librería ROOM para gestionar las operaciones de base de datos
- RNF-03: La aplicación utilizará el color negro para los valores medidos en tiempo real por cada sensor
- RNF-04: La aplicación utilizará el color verde para el valor máximo medido por cada sensor
- RNF-05: La aplicación utilizará el color rojo para el valor mínimo medido por cada sensor
- RNF-06: La aplicación utilizará el color verde para la gráfica de tiempos reales medidos durante la ejecución de los casos de prueba de operaciones pesadas realizadas con secuencias
- RNF-07: La aplicación utilizará el color amarillo para la gráfica de tiempos medios medidos durante la ejecución de los casos de prueba de operaciones pesadas realizadas con secuencias
- RNF-08: La aplicación utilizará el color rojo para la gráfica de tiempos reales medidos durante la ejecución de los casos de prueba de operaciones pesadas realizadas con listas
- RNF-09: La aplicación utilizará el color azul para la gráfica de tiempos reales medidos durante la ejecución de los casos de prueba de operaciones pesadas realizadas con listas

- RNF-10: La aplicación deberá ser compatible con la versión mínima de Android Lollipop (5.0) SDK Version 21
- RNF-11: La aplicación deberá seguir las Material Android Design Guidelines (12)
- RNF-12: La aplicación utilizara el sistema de control de versiones de Git alojado en Github (13)
- RNF-13: La aplicación deberá utilizar la librería Koin (14) para gestionar la inyección de dependencias

4. Desarrollo

4.1 Arquitectura

La arquitectura elegida para el desarrollo de la aplicación ha sido Clean Architecture (14), este tipo de arquitectura permite separar el código de la aplicación en diferentes módulos o capas. Estas capas tienen restricciones para comunicarse con el resto, creando así una única dirección en el flujo de la aplicación y permitiendo desacoplar los módulos entre si

Los modulo o capas son los siguiente:

- App:
Contiene todo lo relacionado con las vistas y elementos gráficos de la aplicación, así como las acciones que puede realizar el usuario sobre ellas. Para implementar esta capa se ha utilizado un patrón MVVM (modelo, vista, modelo de la vista), muy similar al MVC clásico (modelo, vista, controlador)

El modelo, al igual que en MVC, representa la capa de datos y la lógica de negocio de la aplicación y para obtener una mayor independencia entre módulos se ha implementado en el módulo de Dominio

La vista presenta la información al usuario y reacciona ante los cambios que ocurran en el modelo a través de los casos de uso, similar a un patrón MVC activo

El modelo de la vista (ViewModel) es un actor intermediario entre el modelo y la vista, contiene toda la lógica de presentación

La vista en MVC depende del modelo y del controlador para obtener datos o modificarlos. Si hay un cambio en la vista que implique un cambio en el modelo necesita acceder al controlador y si hay datos nuevos, depende del modelo para presentarlos, en cambio, en el caso del MVVM, la vista solo depende del modelo de la vista (ViewModel) para obtener o modificar los datos, además nadie va a tener que decir a la vista que se actualice como en un MVC pasivo ya que ella misma puede observar los cambios directamente

El modelo en MVC puede tener demasiadas responsabilidades como obtener los datos de diferentes fuentes, informar al controlador sobre cambios y preparar datos para que se puedan mostrar en la vista. En MVVM el modelo queda

totalmente desacoplado de la vista y solo contiene información, nunca información para manipularla.

En MVC no queda tampoco muy claro quien se debería encargar de la lógica de presentación ya que el modelo solo debería enviar datos a la vista y, por otro lado, llevar esta lógica a la vista complicaría algunos test. En MVVM, esta responsabilidad está muy clara y recae sobre el modelo de la vista (ViewModel)

También hay que aclarar que en la programación de esta aplicación móvil se utilizará un único "Activity" como coordinador de todas las vistas y su función será únicamente la de configurar cada "Fragment" y mostrarlo. Cada "Fragment" tendrá después su lógica de vista encapsulada dentro de el

Esta capa App solo se podrá comunicar con la capa de Domain, a través de los ViewModel para ejecutar casos de uso de la aplicación

- Domain:

Incluye todos los casos de uso que tiene que ser capaz de ejecutar la aplicación, esta capa recibe llamadas directamente desde los ViewModel que a su vez reciben las acciones que el usuario puede desencadenar. Estas acciones pueden ser "activas", donde el usuario por ejemplo presiona un botón o un campo de texto o acciones "pasivas" como por ejemplo la navegación a una nueva vista.

Los casos de uso están implementados con Corrutinas para evitar bloquear las demás acciones que se lleven a cabo en el hilo principal de la aplicación, mientras se realizan las tareas de forma asíncrona

Por último, esta capa solo puede comunicarse con la capa Data a través de los Repository.

- Data

Corresponde a la implementación de las reglas de negocio previamente definidas en la capa de Dominio. La mayor parte de la lógica va a consistir simplemente en la solicitud y persistencia de datos

Lo más importante de esta capa son los Repository, estos objetos implementan el patrón repositorio para obtener y enviar datos, esto quiere decir que, para una determinada operación, es capaz de decidir dónde y cómo encontrar la información necesaria, ya sea en la base de datos local, en un servidor web, en los sensores del dispositivo o en cualquier otro productor de datos

Esta capa solo puede comunicarse a través de los DataSources con la capa Database, con la capa Sensors y con la capa Collections

- Database:

Implementa toda la gestión de comunicación con la base de datos del dispositivo y las operaciones CRUD para poder leer, guardar, editar y borrar datos. Se utiliza un patrón DAO para el acceso a los datos, este patrón propone encapsular completamente la lógica para acceder a los datos, de esta forma el DAO proporcionara los métodos necesarios para realizar las operaciones

- Sensors:

Implementa toda la gestión de datos recogidos por los sensores del dispositivo, así como la conexión con los mismos, es una capa sencilla que se ha implementado de esta manera para que pueda ser usada como otro productor de datos diferente, gracias a la encapsulación, la aplicación no conoce si estos datos vienen de un sensor del móvil o de cualquier otra fuente lo que permitiría cambiarlos por cualquier otro tipo de productor de datos en tiempo real

- Collections:

Implementa toda la gestión, ejecución y medición de tiempo de las operaciones pesadas realizadas sobre Listas y Secuencias, al igual que los sensores, se ha encapsulado de esta forma para poder ser utilizada como otro productor de datos totalmente independiente

Como resumen, el flujo de una acción del usuario como, por ejemplo, borrar nuevo equipo de la base de datos (cualquier otra operación o caso de uso ejecutado seguirá el mismo flujo) sería el siguiente:

1. El usuario presiona el botón de borrar un equipo
2. La vista envía esa acción al ViewModel
3. El ViewModel ejecuta el caso de uso borrar equipo
4. El caso de uso llama al repositorio de equipos para que lo borre
5. El repositorio escoge el datasource correspondiente, en este caso la base datos y avisa de que hay que borrar el equipo
6. El datasource se comunica con el DAO que borrar el equipo de la base de datos
7. El datasource avisa de nuevo al repositorio de que se ha realizado la operación con éxito
8. El repositorio hace lo mismo con el caso de uso
9. El caso de uso se comunica con el ViewModel
10. El ViewModel cambia su estado a operación completada
11. La vista que observa al estado del ViewModel detecta el cambio y muestra al usuario que la operación fue completada con éxito

A continuación, se muestra el diagrama de paquetes de la aplicación en forma de diagrama, el paquete App se comunica con el paquete Domain a través del subpaquete UseCase, el paquete de Data se comunica con los paquetes Sensors, Collections y Database a través del subpaquete Datastore

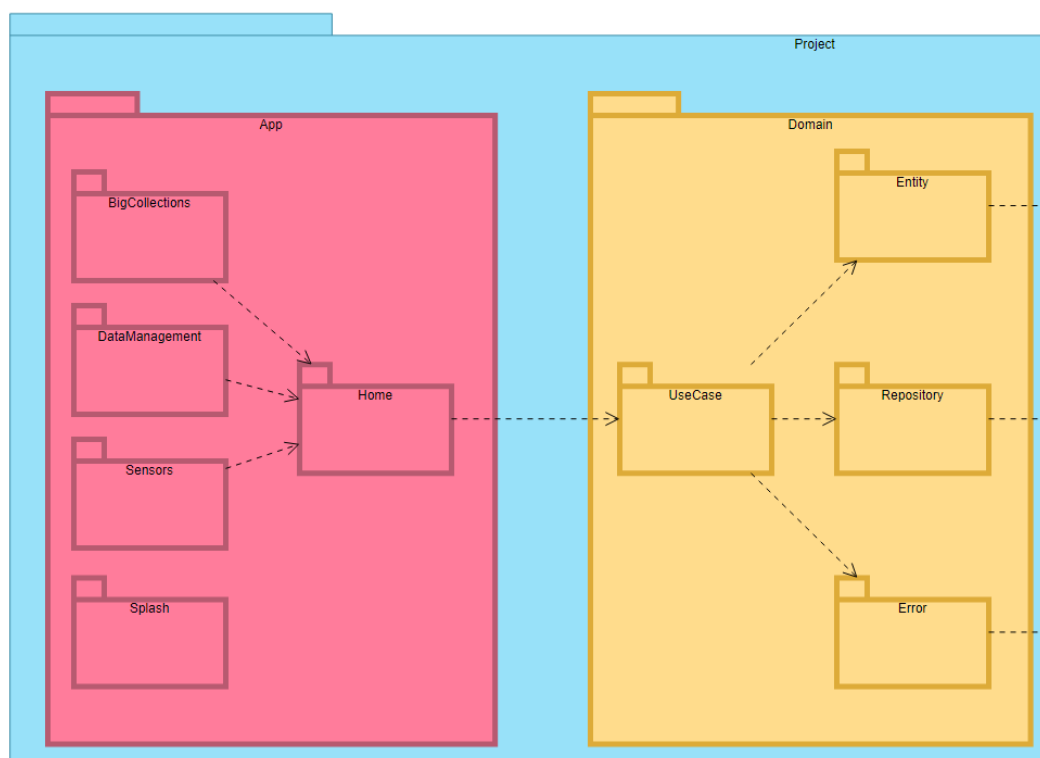


Figura 4-1 Project Package Model 1

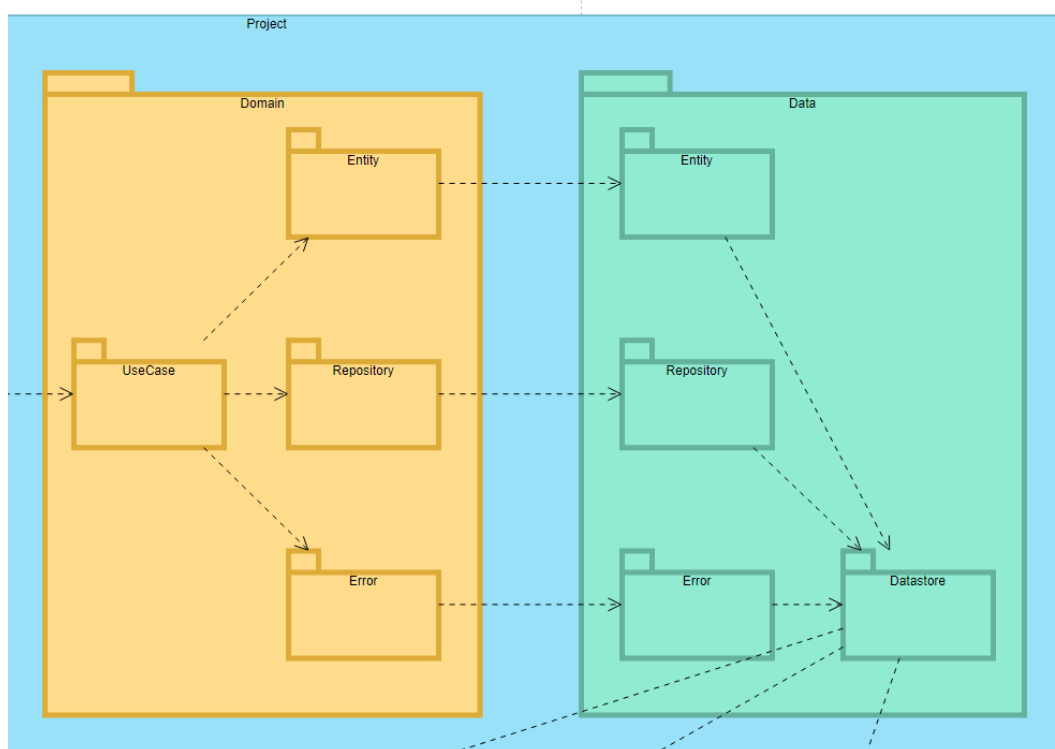


Figura 4-2 Project Package Model 2

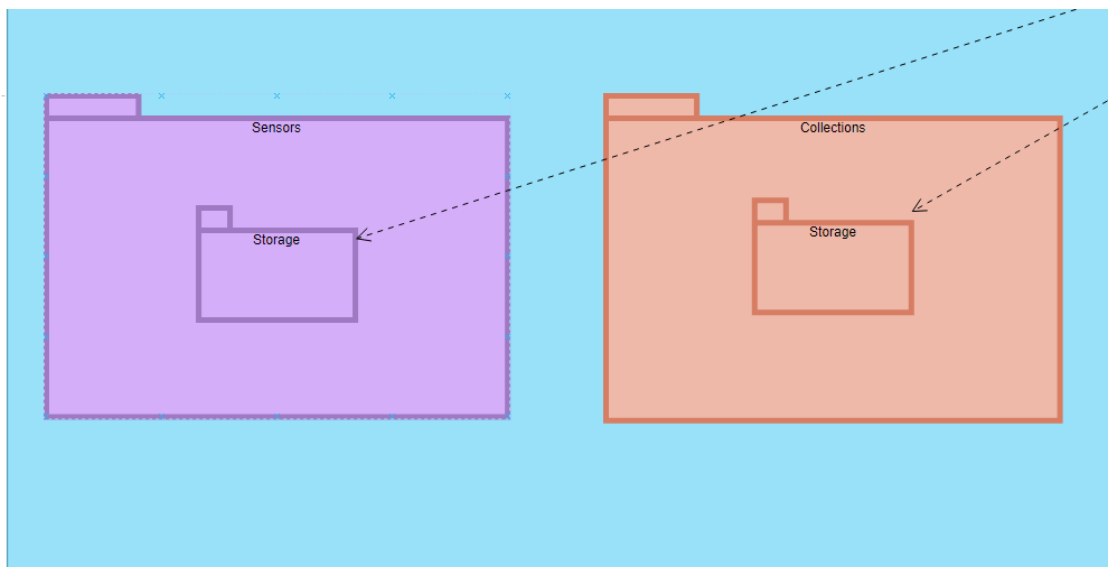


Figura 4-3 Project Package Model 3

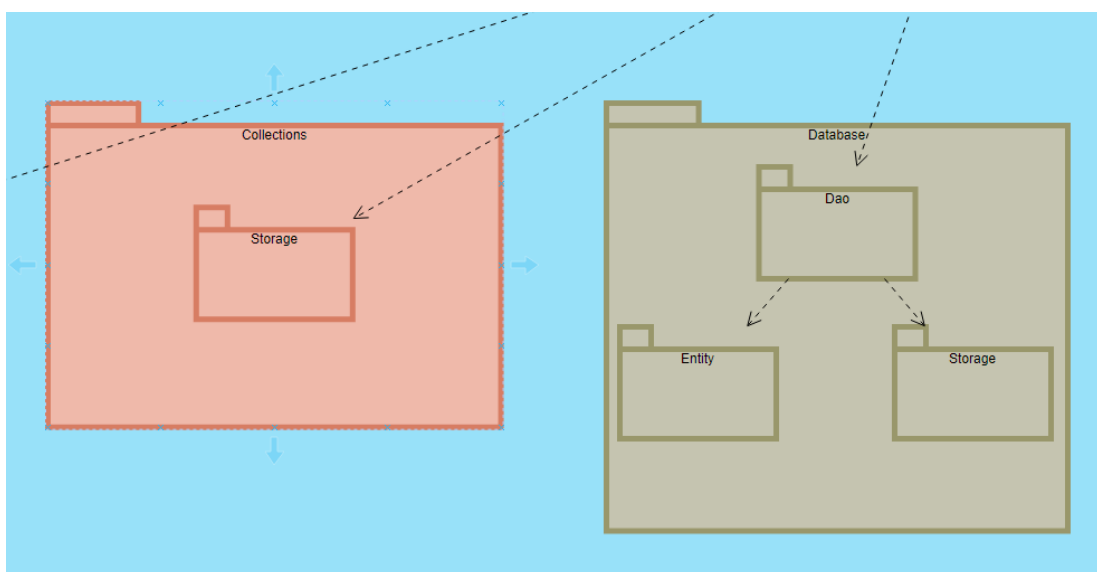


Figura 4-4 Project Package Model 4

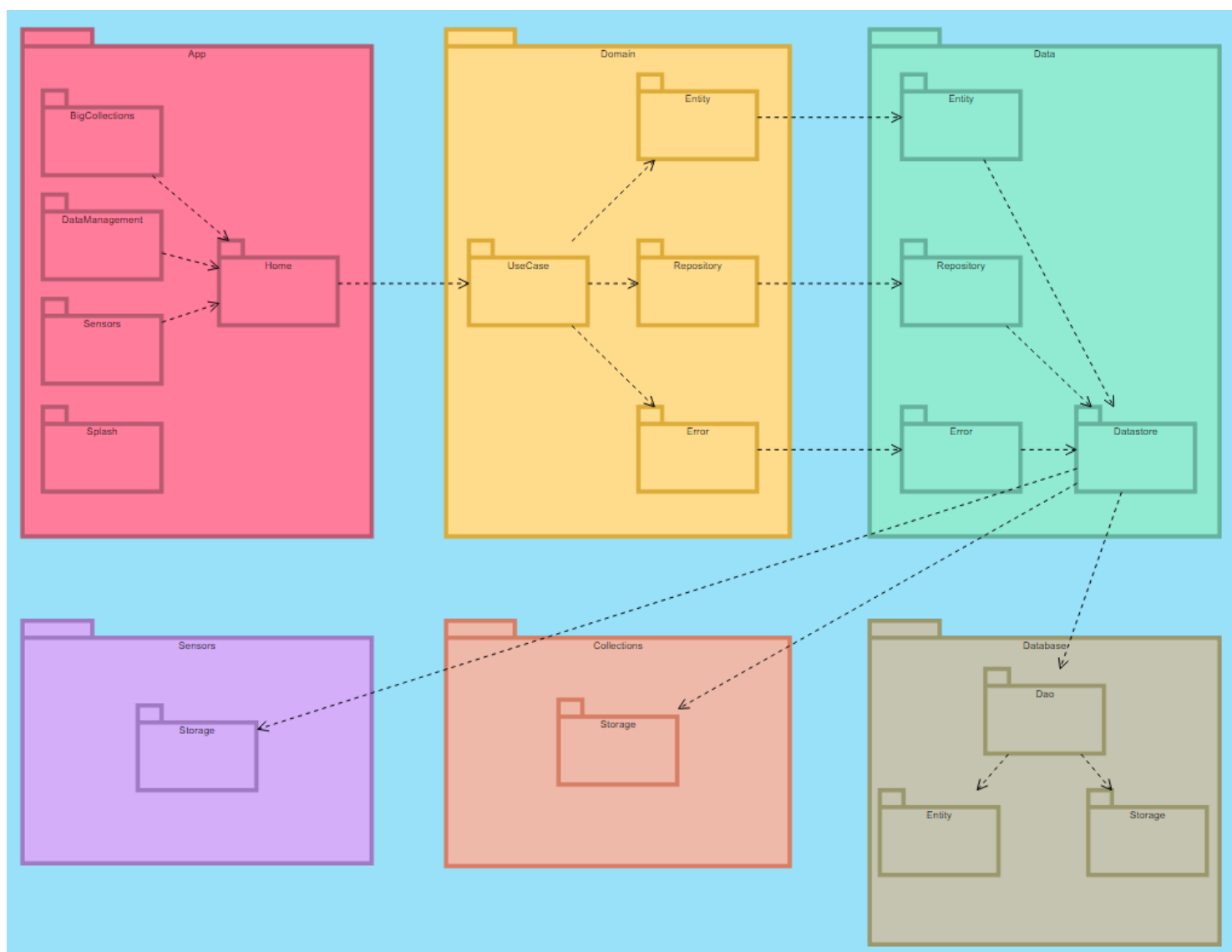


Figura 4-5 Project Package Model General

4.2 Otros aspectos importantes

Se han utilizado tipos de datos un poco especiales para la implementación, además de los ya mencionados, que hay que destacar:

- **Option<A>:**

Este tipo de datos puede contener un objeto de tipo A (Genérico) o estar vacío que se representa con “None”, es muy útil en operaciones que pueden fallar y que además no devuelven un resultado o su resultado no nos es útil, por ejemplo, guardar un objeto en la base de datos puede tener éxito o lanzar una excepción, en ese caso usaríamos un `Option<Exception>` y sabríamos que si su valor es `None` todo ha ido bien pero que si su valor es un objeto de tipo `Exception` habrá ocurrido algún error (16)

- **ViewState:**

Todos los `ViewModel` definen su propio `ViewState` que no es más que una sealed class con la que se crean otras clases y objetos que heredan de ella para definir y establecer los posibles estados que puede tener el conjunto Vista-`ViewModel`. Las vistas siempre observan el estado del `ViewModel` que cambiara en determinadas situaciones y ellas reaccionaran a ese cambio. Por ejemplo, en el caso anterior cuando un usuario quiere borrar un equipo, la vista recoge esa acción y se la da al `ViewModel`, este cambia de estado a “Realizando Operación” y la vista ante este cambio muestra una barra de progreso, cuando la operación termina el `ViewModel` cambia su estado a “Operación exitosa” y la vista reacciona y muestra un mensaje al usuario

5. Resultados y conclusiones

5.1 Operaciones sobre bases de datos

Cuando se realizan operaciones sobre bases de datos tenemos que diferenciar entre dos tipos, las bases de datos que van a cambiar con mucha frecuencia y las que van a permanecer más estables. En cuanto a eficiencia y tiempo de ejecución no hay ninguna diferencia notable entre utilizar Flow y Corrutinas o Listas y tareas asíncronas, sí que es cierto que en cuanto a código es más sencillo y simple utilizar Flow y Corrutinas ya que gestionan automáticamente los hilos y datos.

La principal diferencia reside en que con Flow vamos a tener siempre el estado actualizado de la base de datos y si esta va a cambiar con frecuencia o por el carácter de nuestra aplicación necesitamos hacer muchas consultas, es mucho mejor utilizar la programación reactiva, en cualquier otro caso va a ser indiferente utilizar la programación funcional o la reactiva.

Se puede observar fácilmente como la lista de datos obtenidos de forma reactiva está siempre actualizada con cualquier cambio que se produzca desde el primer momento en que se piden los datos, si se crea, borra o actualiza un equipo, se emite un evento a través del canal abierto y se actualiza la vista.

En cambio, al utilizar listas normales, cuando se piden los datos el canal de datos queda cerrado por lo que la vista no sabe cuándo ha habido una actualización hasta que pide los datos de nuevo de forma manual y se actualiza la vista de nuevo, esto provoca que los datos que visualiza el usuario puedan no corresponderse con los almacenados en la base de datos en todo momento

En las siguientes imágenes se puede dicho comportamiento, siendo la columna de la derecha la lista de datos obtenidos de forma Funcional y la columna de la izquierda la lista de datos obtenidos de forma Reactiva

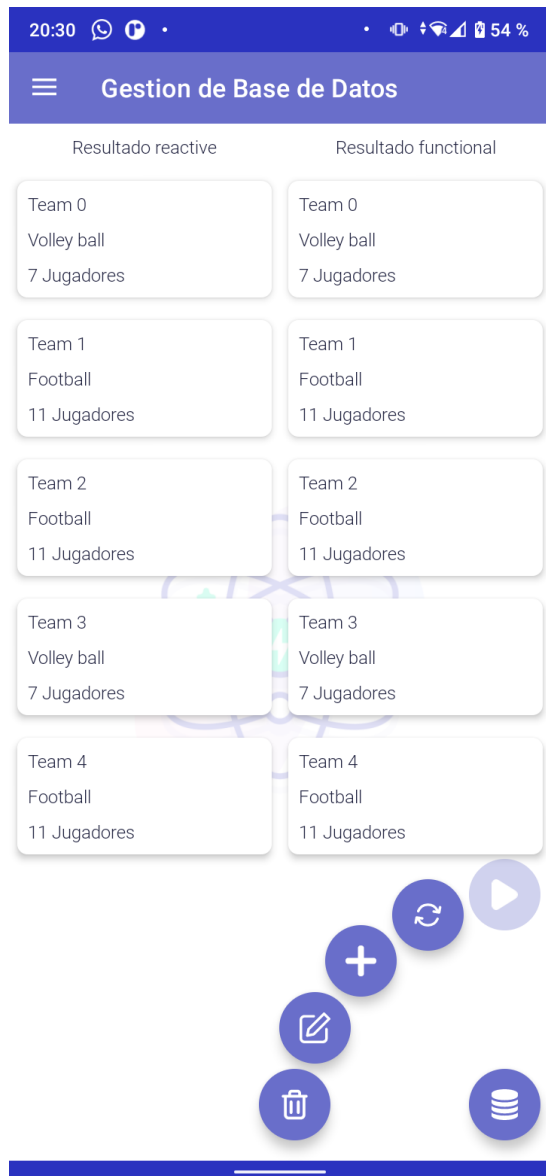


Figura 5-2 Database Read

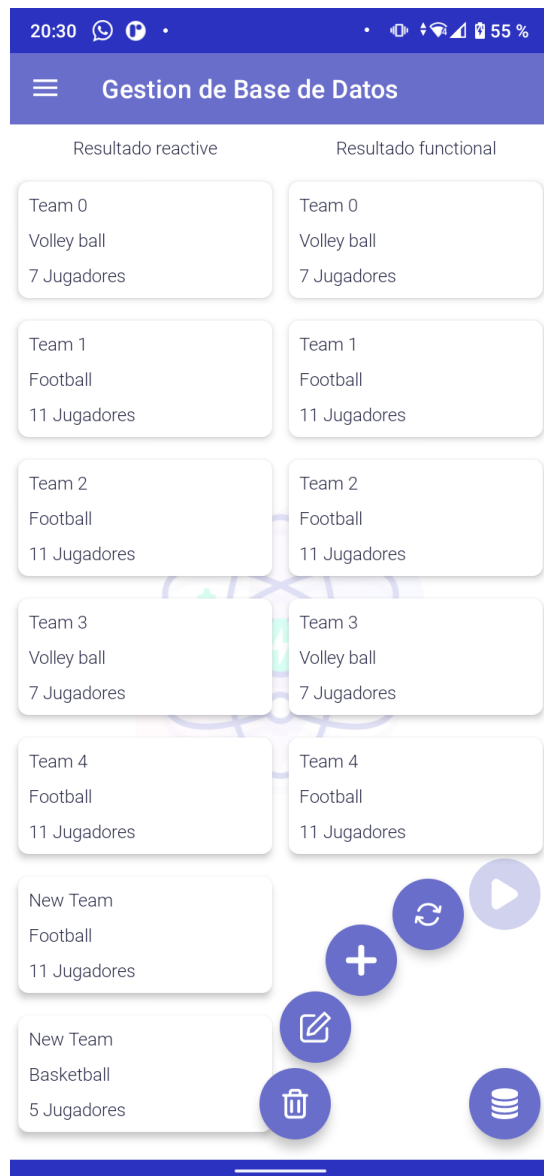


Figura 5-1 Database Create Team

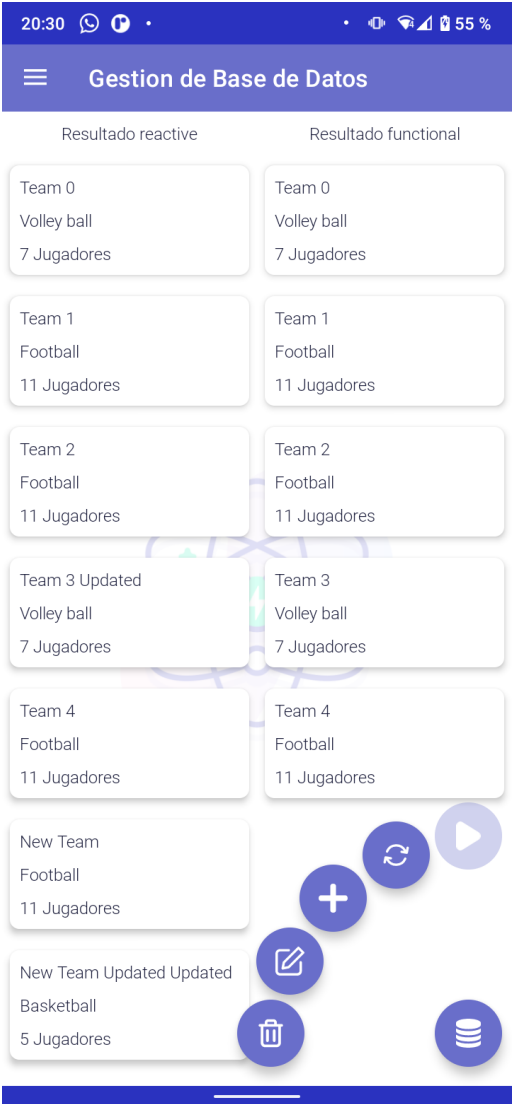


Figura 5-3 Database Update Team

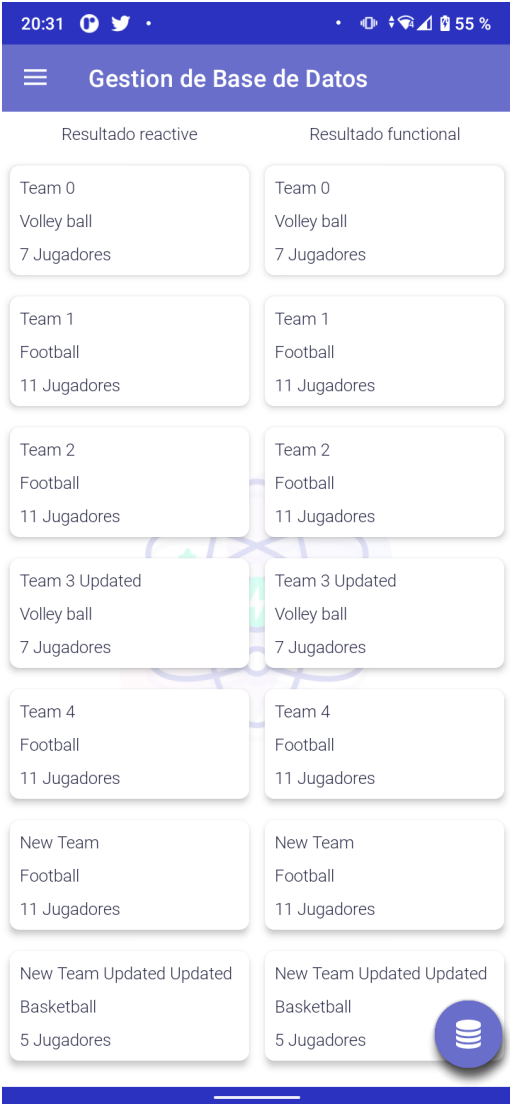


Figura 5-4 Database Synchronize

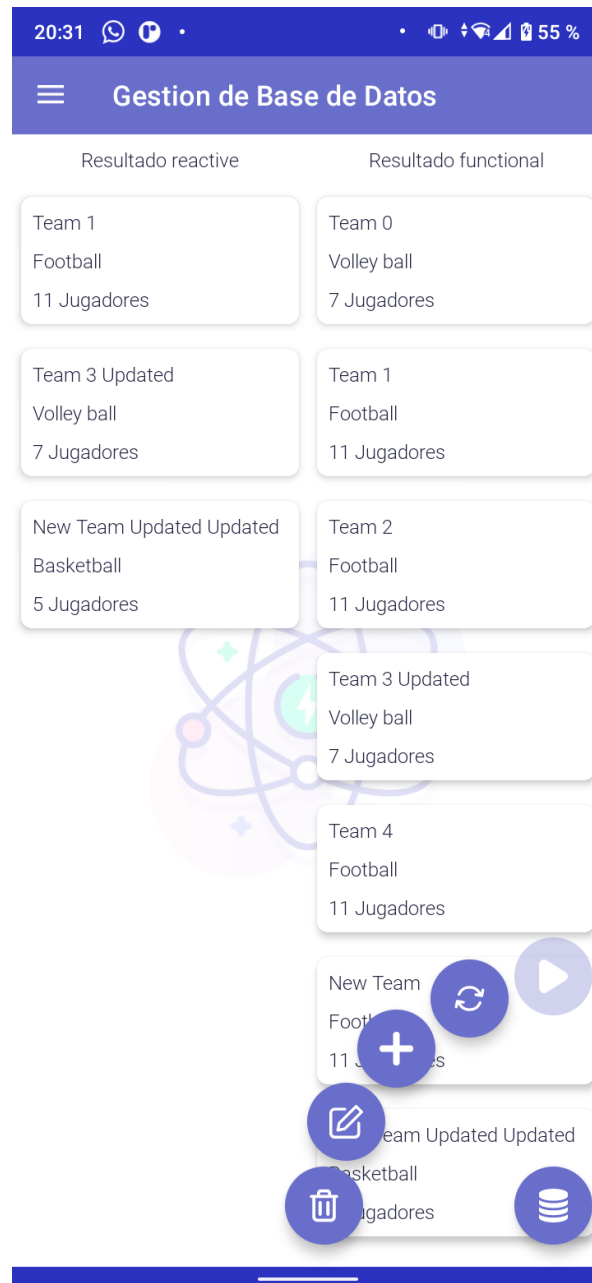


Figura 5-5 Database Delete Team

5.2 Datos en tiempo real

Cuando queremos recoger datos en tiempo real y trabajar con ellos, como por ejemplo en este caso, los sensores de luminosidad, orientación y aceleración, la programación funcional presenta un grave problema que es el intervalo de consulta. Este intervalo es el tiempo que va a pasar entre dos consultas consecutivas, si el intervalo de tiempo es muy grande vamos a perdernos muchos valores, dependiendo del carácter de los datos, aplicación, caso de uso, etc. esto puede suponer un grave problema. En cambio, si el intervalo es muy pequeño, probablemente consigamos no perder ningún dato, pero vamos a estar realizando muchas más consultas de las necesarias, si es una aplicación pequeña que no tiene grandes cargas de trabajo puede que no notemos la diferencia, pero realizar esta tarea en segundo plano en una aplicación pesada con muchos flujos de datos va a suponer un problema en cuanto a eficiencia de esta.

Es muy interesante observar cómo poco a poco los valores máximos y mínimos comienzan a ser diferentes para ambos tipos de programación ya que en la reactiva vamos a tener siempre los valores correctos y en la funcional puede ocurrir que se genere un valor, no coincida con el intervalo de tiempo de consulta, se genere otro y se produzca la consulta después, habiendo perdido ese valor intermedio

Para ver esto mejor incluso, se le permite al usuario jugar variando el intervalo de consulta para realizar un Polling muy rápido y constante o mucho más lento, en este caso es interesante también fijarse en la efectividad de lectura de cada método, para ello se contabiliza el número de consultas realizadas y se divide entre el número de datos generados, obteniendo así la efectividad de la medida y poder observar el porcentaje de acierto.

Se puede tratar de medir o estimar el intervalo adecuado realizando varias pruebas variando el tiempo de consulta y comprobando cual es el más eficiente, esto supone gastar tiempo de desarrollo en ello y a veces puede ser complicado de calcular.

Este problema queda completamente resuelto cuando utilizamos la programación reactiva ya que cada dato recogido por los sensores va a ser emitido por el canal de datos abierto, no necesitamos intervalos de consultas ya que realmente no vamos a hacer ninguna consulta, será el propio dato generado quien nos avise de que ya está listo, de esta manera no vamos a perder ningún dato ni a desperdiciar tiempo de ejecución y recursos en hacer más consultas de las necesarias

En la programación reactiva siempre va a ser del 100% ya que cada dato recogido por el sensor es emitido por el canal de datos mientras que, en la funcional, si ponemos un intervalo muy lento, el porcentaje de efectividad bajara mucho y si ponemos un intervalo muy rápido, sobrepasara el 100% ya que estaremos haciendo muchas más consultas, iremos más rápido consultando que los sensores recogiendo datos, lo cual también es muy ineficiente

En las siguientes imágenes podemos observar dicho comportamiento y resultados



Figura 5-7 Sensors Running



Figura 5-6 Sensors High Interval



Figura 5-8 Sensors Low Interval

5.3 Secuencias y listas

En este caso la mejor forma de realizar la comparación entre Listas y Secuencias ha sido representar los tiempos de ejecución de todos los casos de prueba en forma de grafica. Se ha medido de dos formas diferentes:

1. Tiempo real en milisegundos:

Es el tiempo real que se ha tardado en ejecutar todas las operaciones sobre la Lista o la Secuencia, se empieza a medir justo antes de realizar la primera operación, el tiempo que se tarda en la creación de la propia Lista o Secuencia no se tiene en cuenta. Se termina de medir justo después de realizar la última operación.

2. Tiempo medio en milisegundos:

Se calcula la media de tiempos reales entre la ejecución actual, la anterior y la siguiente (en la primera y última ejecución se usa solo el actual y el siguiente o anterior), de esta forma podemos suavizar un poco los picos de la gráfica y hacernos una idea más general de cómo cambia el tiempo de ejecución en los diferentes casos de prueba.

Después de realizar todas las ejecuciones de los casos de prueba donde hemos ido variando diferentes parámetros, observamos como claramente el tiempo de cada prueba es superior en las Listas frente a las Secuencias que tienen un tiempo de ejecución más bajo, es importante resaltar que para las Secuencias es más constante y solo hay un cambio brusco cuando pasamos de 1000 a 10.000 elementos para comprobar si son primos o no, esto quiere decir que no influye el tamaño total de la secuencia (puede ser infinita incluso) si no el número de elementos sobre los que realizamos las operaciones ya que las operaciones se realizan elemento a elemento y no sobre toda la Secuencia.

Para las Listas en cambio la gráfica tiene muchos más picos y es mucho menos constante, cada tiempo de ejecución depende prácticamente de los tres parámetros que utilizamos ya que tiene que crear listas intermedias entre operaciones e ir resolviéndolas poco a poco, realizando las operaciones para la lista entera, aunque solo necesite un número determinado de elementos.

Podemos observar en las siguientes imágenes dicho comportamiento y resultados

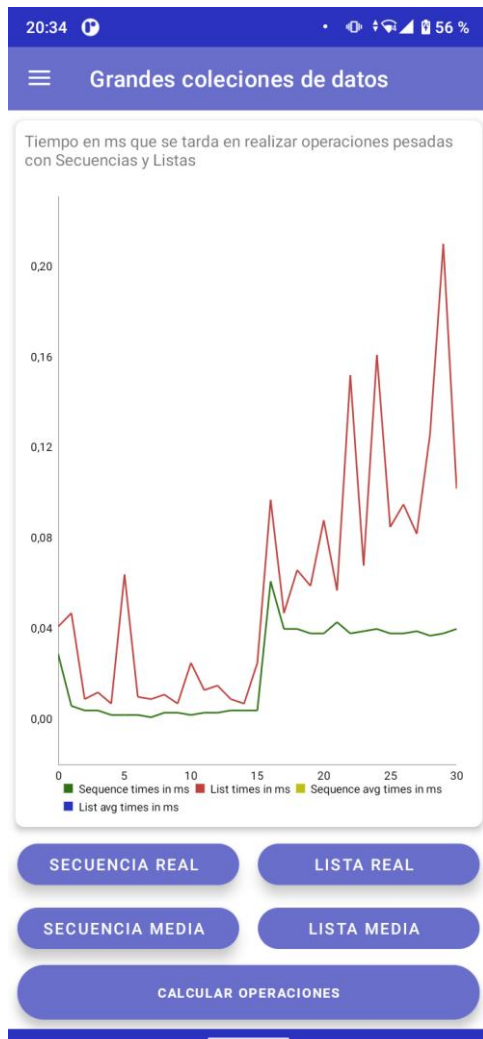


Figura 5-9 Collections One Execution

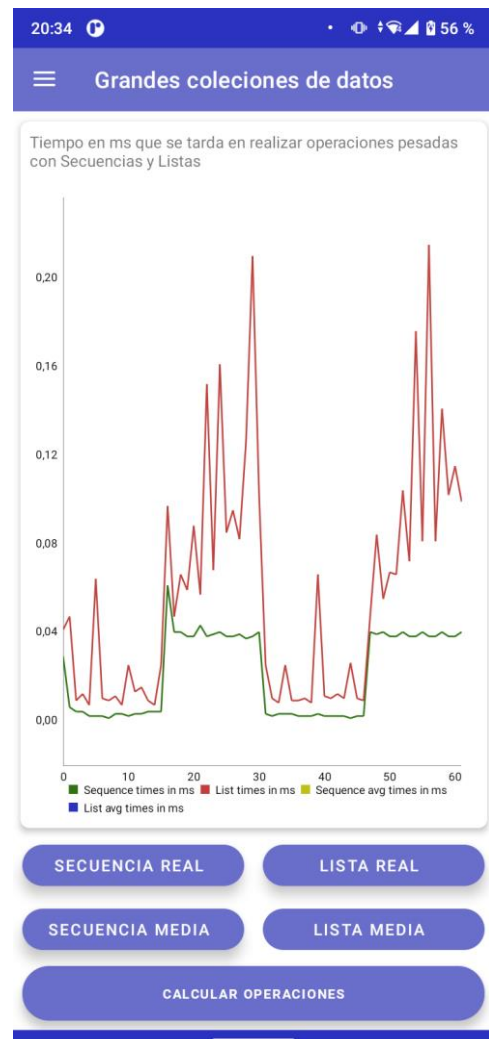


Figura 5-10 Collections Two Executions

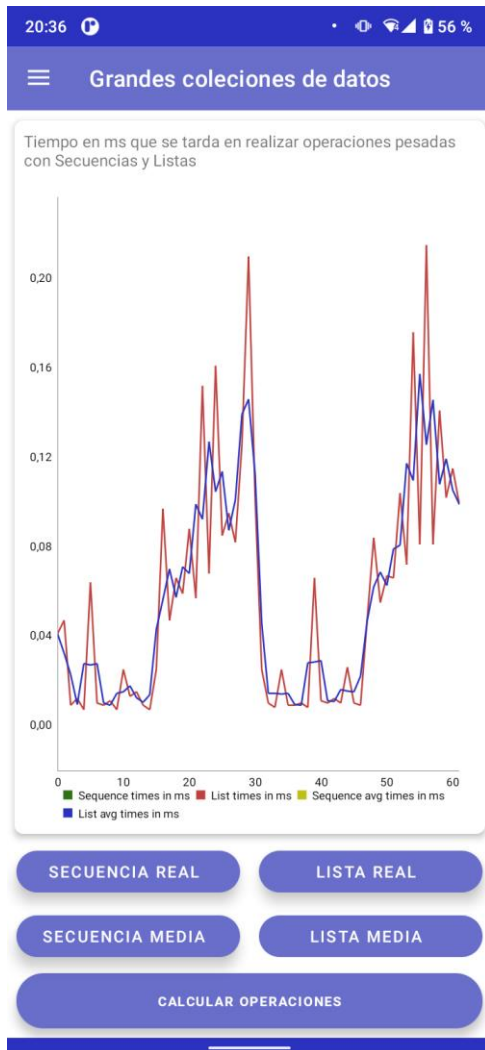


Figura 5-11 Collections List Data

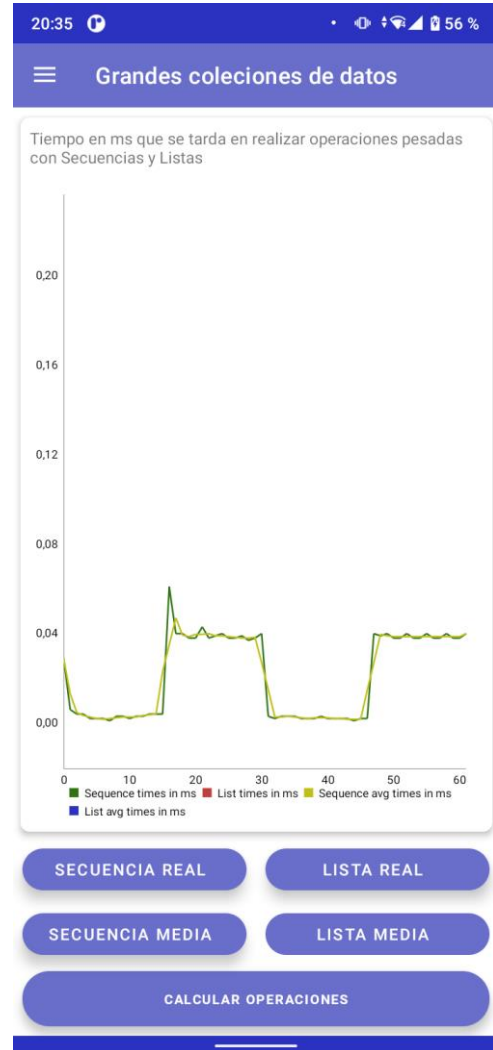


Figura 5-12 Collections Sequence Data

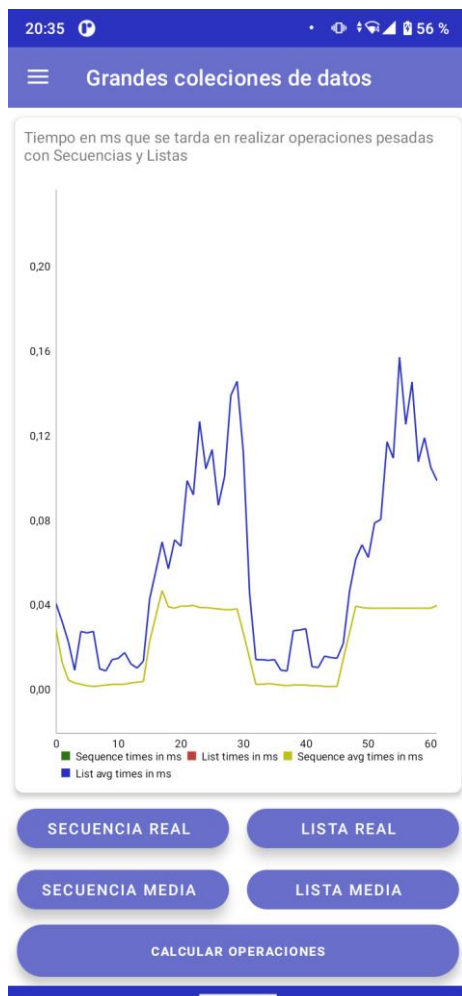


Figura 5-13 Collections Avg Data

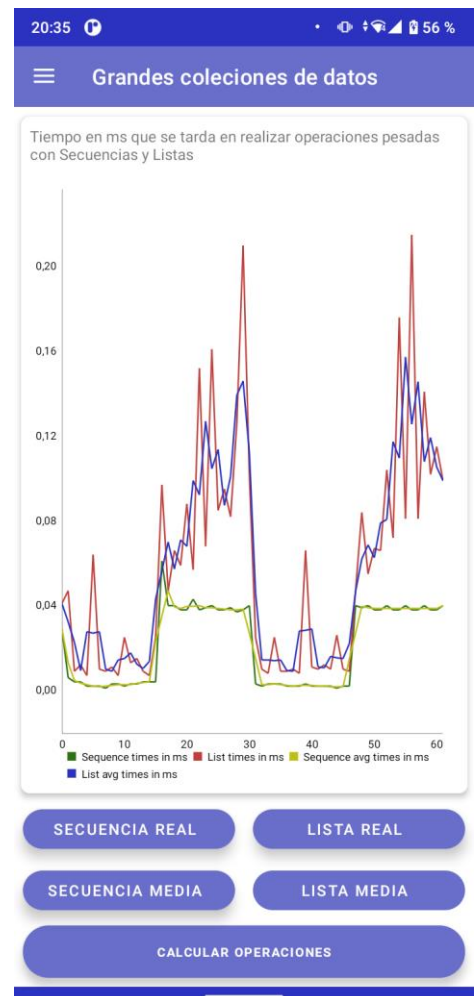


Figura 5-14 Collections All Data

Por último, es muy importante saber cuándo utilizar Listas y Secuencias, aunque pueda parecer que las Secuencias son más rápidas y eficientes no siempre es así, cuando es un número pequeño de operaciones o las colecciones de datos no son bastante grandes, es mucho más rápido y eficiente trabajar con Listas normales

6. Líneas de trabajo futuro

El trabajo futuro podríamos dividirlo en dos líneas diferentes, investigación y estudio de datos

6.1 Investigación

En la línea de investigación me gustaría descubrir y explorar nuevos usos de la programación reactiva, como por ejemplo tratar de implementarla con llamadas a una API en vez de a la base de datos local. Podría implementarse en un chat donde se utilizase para enviar y recibir mensajes o en una aplicación de consulta de valores de divisas donde estos datos están actualizándose constantemente

También me gustaría refinar el uso de la programación reactiva en la arquitectura de aplicaciones creando una librería que gestione los cambios de estados de las vistas provocados por los ViewModel y las acciones del usuario, donde estas vistas se suscriban a esos eventos y reaccionen de forma correcta, algo como lo que se implementa en este proyecto, pero más refinado y fácil de usar

6.2 Estudio de datos

En la línea del estudio de datos habría que centrarse en las diferencias en el uso de la memoria entre la programación funcional y reactiva y sobre todo en el uso de Secuencias o Listas, en este proyecto nos hemos centrado sobre todo en el estudio del tiempo de ejecución de operaciones y la efectividad de la recogida de datos, nos faltaría ver las diferencias en cuanto al uso de recursos y comparar ambos métodos

Bibliografía

1. **Leiva, Antonio.** Universidad Pontificia de Salamanca. [En línea] <http://www.upsa.es>.
2. **Cifuentes Daniel, Zaldua Alejandra.** Paradigmas de Programación. [En línea] https://ferestrepoca.github.io/paradigmas-de-programacion/progfun/funcional_teor%C3%ADa/index.html
3. **Delgado, Pedro.** Future Space. [En línea] <https://www.futurespace.es/reactividad-en-servicios-web-java/>.
4. **Bonér Jonas, Farley Dave, Kuhn Roland, and Thompson Martin.** The Reactive Manifesto. [En línea] <https://www.reactivemanifesto.org/>.
5. **Developers, Google.** Android Developers. [En línea] <https://developer.android.com/topic/libraries/architecture/livedata>
6. **Developers, Google.** Android Developers. [En línea] <https://developer.android.com/kotlin/flow>
7. **Developers, Google.** Android Developers. [En línea] <https://developer.android.com/kotlin/coroutines>
8. **Develou.** [En línea] <https://www.develou.com/secuencias-en-kotlin/>.
9. **Developers, Google.** Android Developers. [En línea] <https://developer.android.com/kotlin/flow/stateflow-and-sharedflow>
10. **Developers, Google.** Android Developers. [En línea] <https://developer.android.com/training/data-storage/room>
11. **Developers, Google.** Android Developers. [En línea] https://developer.android.com/guide/topics/sensors/sensors_overview
12. **Google. Material Design.** [En línea] <https://material.io/design/guidelines-overview>
13. **Sanzo, Jorge.** GitHub. [En línea] https://github.com/Jsanzo97/TFM_Reactive_Programming
14. **Martin, Robert C. (Uncle Bob).** The Clean Code Blog . [En línea] <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
15. **Rykov, Mikhail.** Baeldung Kotlin. [En línea] <https://www.baeldung.com/kotlin/koin-di>.

Manual de usuario

En la primera vista que nos aparece nada más ejecutar la aplicación veremos un texto inicial dando un poco de contexto sobre el tema del trabajo, en la parte superior izquierda tenemos un botón para abrir el menú con un icono de tres rayas



Figura 6-1 Home Screen

Al pulsar el botón que abre el menú, se desplegará un menú lateral donde podremos navegar a las diferentes vistas de la aplicación, la vista de gestión de base de datos, la vista de los sensores del dispositivo, la vista de las colecciones de datos y la vista principal donde estamos al comienzo

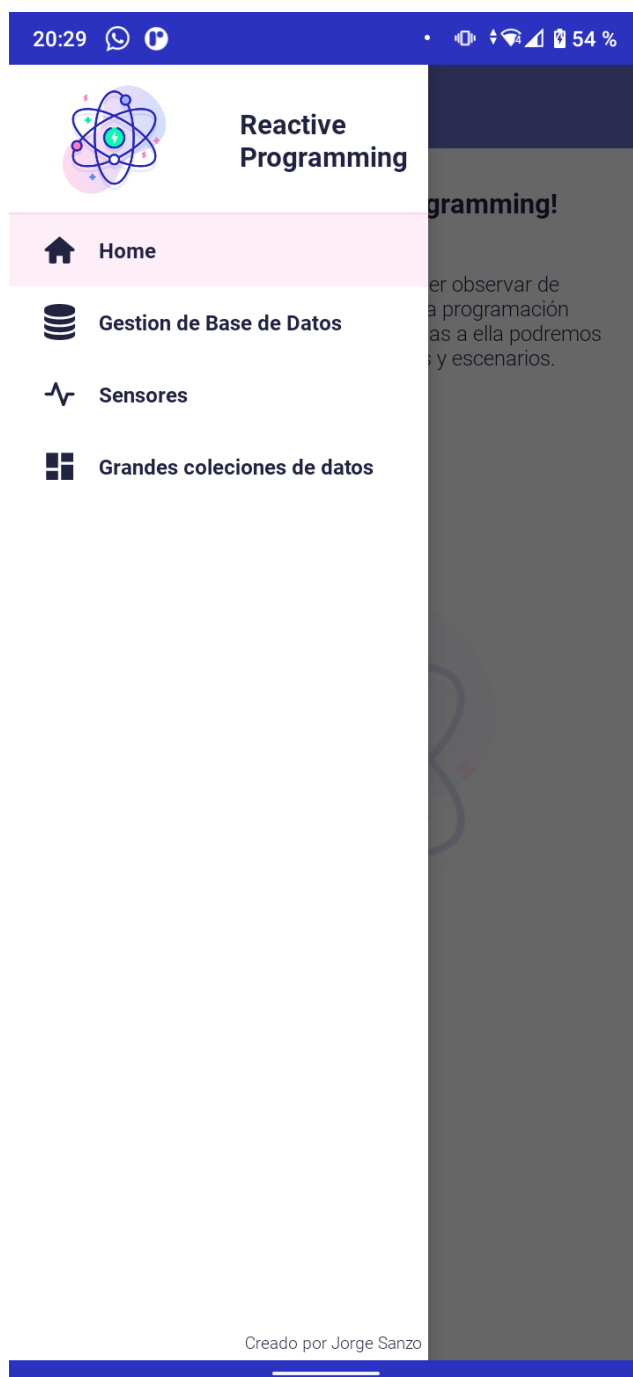


Figura 6-2 Navigation Menu

Si navegamos a la vista de gestión de base de datos veremos un botón abajo a la derecha, este botón desplegara otros cinco nuevos botones

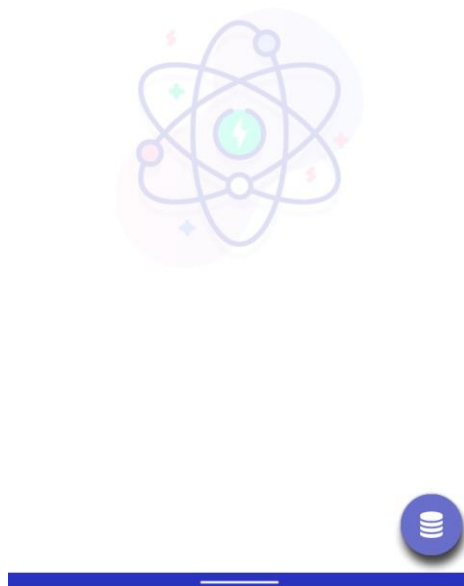


Figura 6-4 Database Screen

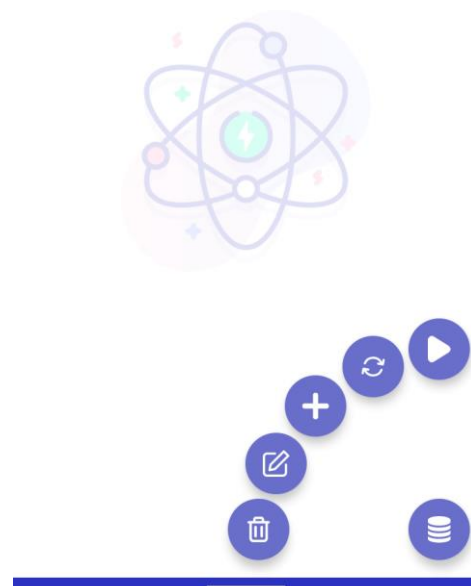


Figura 6-3 Database Menu

De arriba hacia abajo, cada botón hace lo siguiente:

1. Símbolo de "Play", inicia la sincronización con la base de datos y muestra en la columna de la izquierda una lista de equipos obtenidos de forma Reactiva y en la columna de la derecha una lista de equipos obtenidos de forma Funcional
2. Símbolo de "Refresh", pide de nuevo los datos a la base de datos y actualiza ambas listas
3. Símbolo de "Add", crea un equipo con datos aleatorios y lo guarda en la base de datos
4. Símbolo de "Edit", cambia el valor de un campo de un equipo de forma aleatoria
5. Símbolo de "Remove", borra un equipo aleatorio de la base de datos

Si navegamos a la vista de los sensores, podremos observar tres botones y una barra en la parte inferior de la pantalla

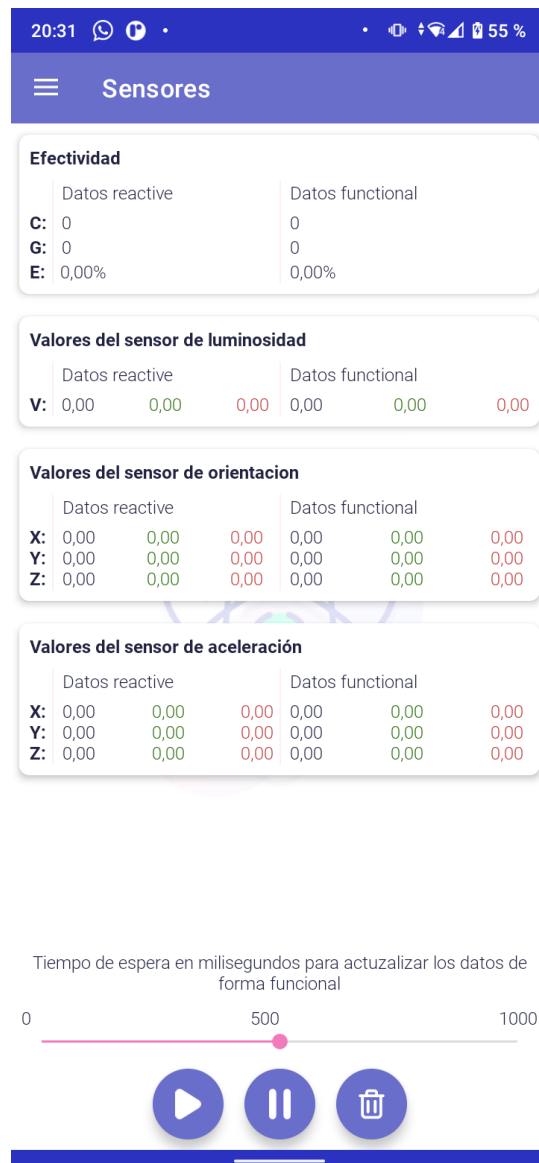


Figura 6-5 Sensors Screen

Su funcionalidad es la siguiente:

1. Símbolo de "Play", inicia los sensores, comienza a recoger los datos y los muestra por pantalla
2. Símbolo de "Pause", pausa los sensores y la recogida de datos de estos
3. Símbolo de "Remove", borra todos los datos recogidos por los sensores
4. La barra establece el intervalo de consulta para la programación funcional, es decir, el tiempo que pasa entre consulta y consulta a cada sensor, el intervalo va desde los 0ms hasta los 1000ms, empezando por defecto en 500ms

En esta misma vista, el texto de color verde indicara el valor máximo para cada sensor, el texto de color rojo el valor mínimo recogido por cada sensor y el texto de color negro indica el valor en tiempo real que se está recogiendo

Por último, si navegamos a la vista de colecciones de datos podremos observar que al principio hay una grafica vacía y un botón en la parte inferior de la pantalla

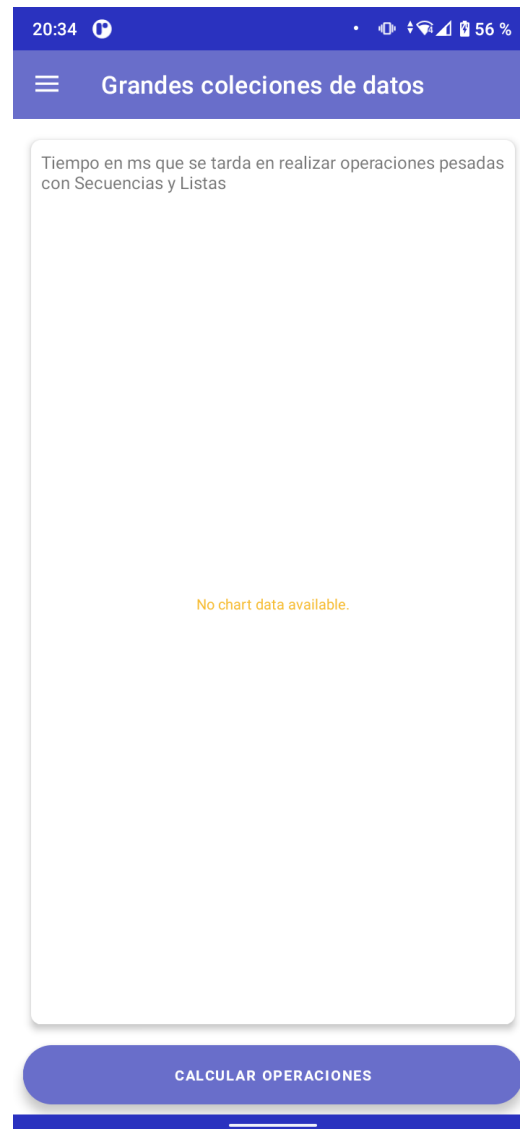


Figura 6-6 Collections Screen

Si presionamos ese botón, lanzaremos todos los casos de prueba con Secuencias y Listas, la primera vez además, se mostraran cuatro botones nuevos con los que podremos ocultar o mostrar por separado los tiempos reales de ejecución de Listas y Secuencias en la gráfica y también sus tiempos medios (Ver Figura 5-9, Figura 5-11, Figura 5-12, Figura 5-13, Figura 5-14). Si volvemos a pulsar el botón de realizar operaciones, los datos de las nuevas ejecuciones se mostrarán a continuación de los anteriores en la misma grafica (Ver Figura 5-10)